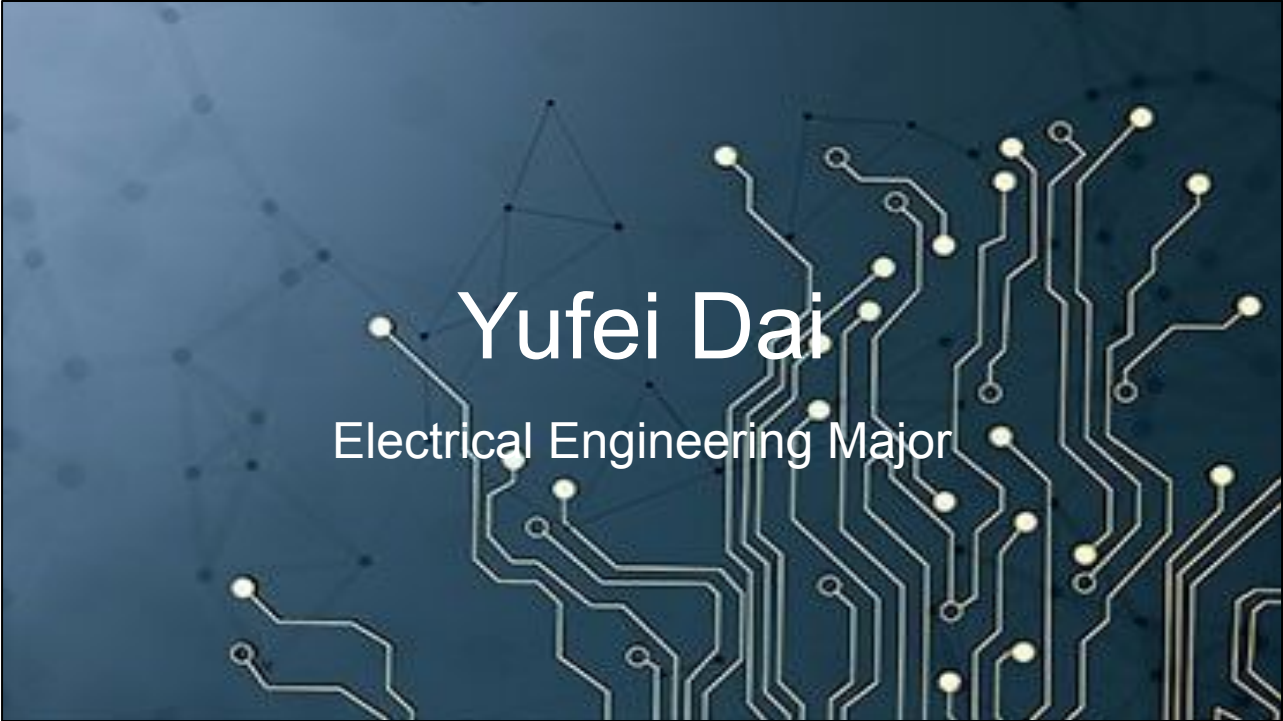


The background of the slide features a dark blue gradient. On the right side, there are intricate white circuit traces that resemble a printed circuit board (PCB). On the left side, there is a faint network diagram consisting of small black dots connected by thin lines, forming a triangular mesh structure.

Yufei Dai

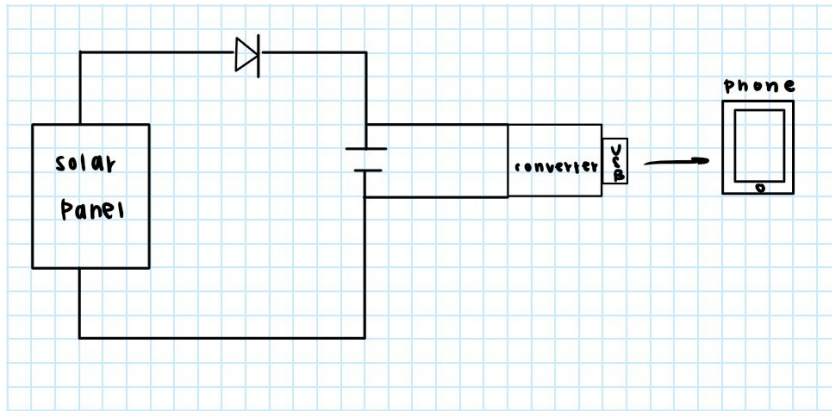
Electrical Engineering Major

[Link to portfolio walk through video](#) (9 min)

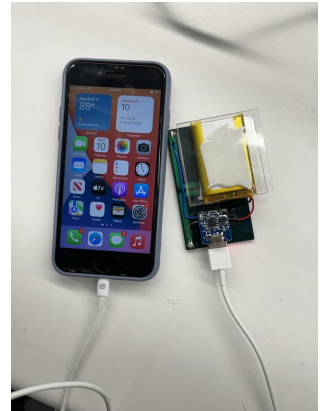
For the complete for all programming projects follow this link to github:
<https://github.com/daiyuf25/Yufei-s-Portfolio>

Solar Panel Phone Charger

Schematic:



Picture:



The solar panel is used to charge a 3.7V battery. The converter increases the battery voltage to 5V for phone charging. The phone only accepts voltages around 5V. A diode is placed between the solar panel and the battery to prevent current from flowing out of the battery and back into the solar panel when it is dark. This is called reverse current which can damage the solar panel.

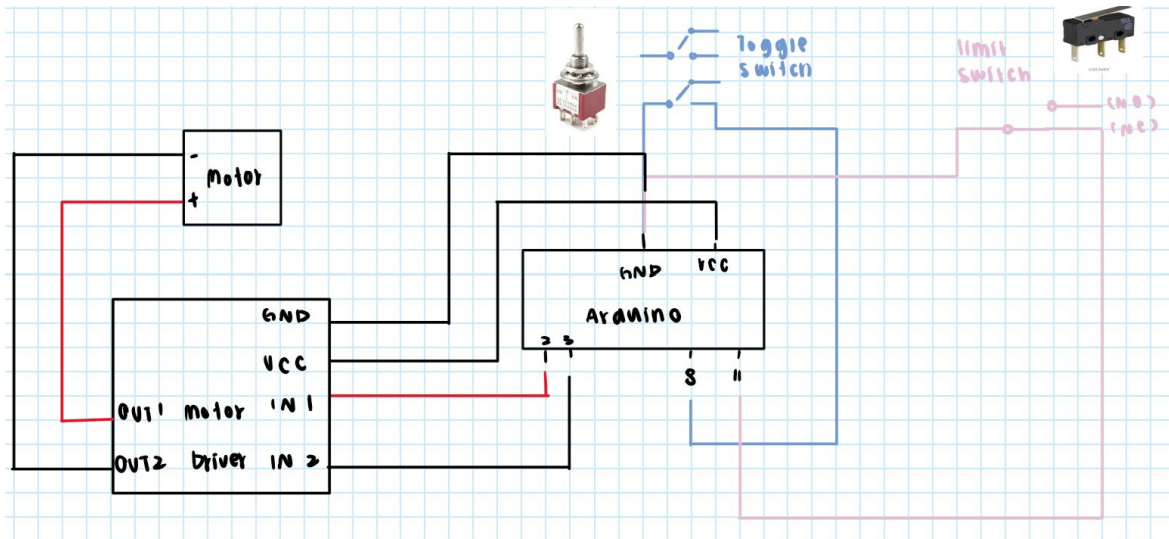
On the right is a picture of the device in action.

Useless Box

Overview: The useless box is a toy where you flip a switch on the box and a finger in the box flips the switch back. I changed the Arduino code to give my useless box a personality. This video is a demo of the the useless box I made.



Electrical Schematic



High Level Overview

- TOGGLE switch and LIMIT switch send signals to the Arduino
- The Arduino uses the signals from the switches to determine signals to send out from pins 2 and 3

Code Overview

Setting up the Arduino Pins

```
void setup() {  
  Serial.begin(115200);  
  Serial.println("Setup complete");  
  // TO DO:  
  // initialize input and output pins  
  pinMode(TOGGLE, INPUT_PULLUP);  
  pinMode(LIMIT, INPUT_PULLUP);  
  pinMode(two, OUTPUT);  
  pinMode(three, OUTPUT);  
  // initialize your initial states  
  state = STOP;  
  //testing state values  
}
```

Loop() Structure

```
void loop() {  
  // TODO read input pins to get status of toggle and limit  
  int TOGGLE_1 = digitalRead(TOGGLE);  
  int LIMIT_1 = digitalRead(LIMIT);  
  
  switch (state) {  
    case STOP:  
      // code  
      break;  
  
    case FORWARD:  
      // code  
      break;  
  
    default:  
      // code  
      break;  
  }  
}
```

setup()

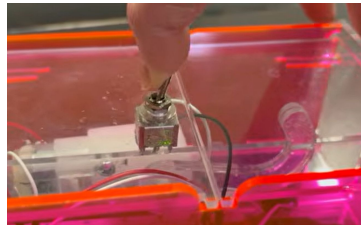
- Set the Arduino pins as input or output
- Sets initial state to STOP

loop()

- The code in inside of the loop is a switch case statement
- There was intended to be a reverse case. However, the code would not enter the reverse case so the reverse case was combined with the forward case.

Stop Case

```
case STOP: // case for STOP
// TO DO: Set your outputs for the motor to stop
Serial.println("Stop");
digitalWrite(two, LOW);
Serial.println(digitalRead(two));
digitalWrite(three, LOW);
Serial.print(digitalRead(three));
// TO DO: Decide whether to change states and update the state variable accordingly
if (TOGGLE_1 == LOW){
    state = FORWARD;
}
break;
```



STOP Case:

- sets both output pins to be LOW
- STOP -> FORWARD: If the user flips the switch, TOGGLE pin will read LOW which switches the state to FORWARD

Forward/Reverse Case

```
case FORWARD: // case for forward
// TO DO: Set your outputs for the motor to move forward
Serial.println("FORWARD");
// TO DO: Decides whether to change states by updating the Transition for your states depending on your input
int now = millis();
int interval = now - start;
Serial.print("start ");
Serial.println(start);
Serial.print("now ");
Serial.println(now);

if (0 <= interval && interval < INTERVAL_SHY) {
  Serial.println("SHY_CASE");
  for (int i = 0; i <= 10; i++) {
    digitalWrite(3, LOW);
    digitalWrite(2, HIGH);
    delay(10);
    digitalWrite(3, LOW);
    digitalWrite(2, LOW);
    delay(30);
    Serial.println("forward");
  }
}
```

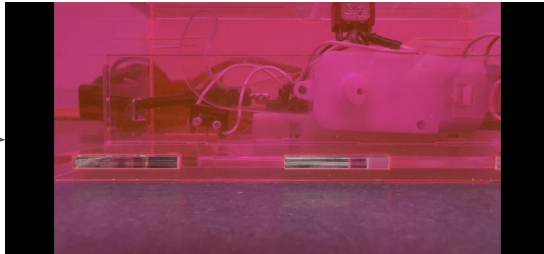
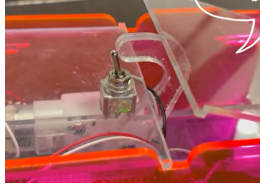
FORWARD Case:

- 1) The signals sent through the output pins depended on how long the program has been running
- 2) Let's look at the shy interval.
 - First the the finger moves counterclockwise at a slow speed.

Forward/Reverse Case

```
//pause for half a sec
digitalWrite(3, LOW);
digitalWrite(2, LOW);
delay(1500);
Serial.println("stop");
//continue moving
for (int i = 0; i <= 20; i++){
  digitalWrite(3, LOW);
  digitalWrite(2, HIGH);
  delay(10);
  digitalWrite(3, LOW);
  digitalWrite(2, LOW);
  delay(50);
  Serial.println("forward");
}
digitalWrite(3, LOW);
digitalWrite(2, HIGH);
Serial.println("forward");
delay(1000);

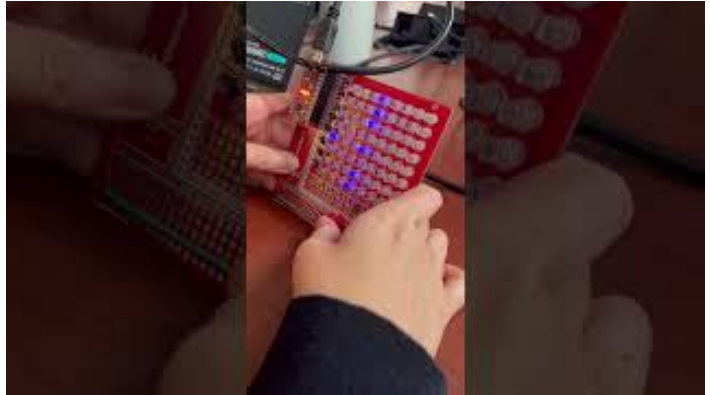
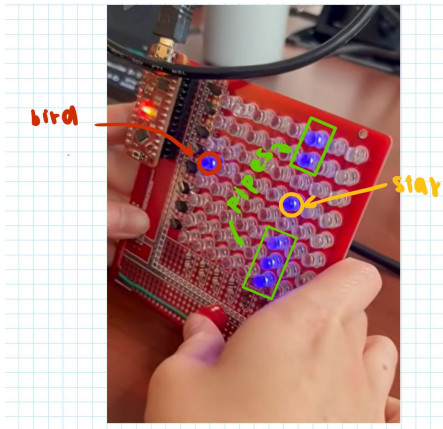
if (digitalRead(TOGGLE) == HIGH){
  Serial.println("reverse");
  digitalWrite(two, LOW);
  digitalWrite(three, HIGH);
  delay(1000);
  if (digitalRead(LIMIT) == HIGH){
    Serial.println("stop");
    state = STOP;
  }
}
```



- The finger then pauses before continuing forward as a quicker pace
- When the finger flips back the toggle switch, TOGGLE reads HIGH which signals the code to tell the finger to reverse
- When the finger hits the limit switch, the LIMIT will read high which will switch the state to STOP

Flappy Bird

Overview: I recreated flappy bird on an 8x8 LED display board. See [here](#) for demo!



Use of Program Duration

```
if ((now - last_bar_shift) > BAR_SHIFT_DELAY){
```

```
//set the last time the pole was shifted to now
```

```
last_bar_shift = now;
```

```
for (byte i = 0; i < 8; i++){
```

```
//find the cathode that is low
```

```
if (check_column (ledOn, 0, i)){
```

```
//turn all of the LEDs in that column off
```

```
for(byte j = 0; j < hole; j++){
```

```
x = j;
```

```
//if you are turning off the zeroth column and the anode pi
```

```
if (x == bird_x and i == 0){
```

```
alive = false;
```

```
}
```

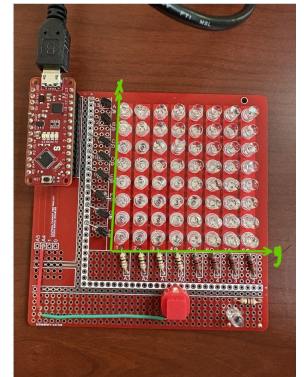
```
y = i;
```

```
ledOn[x][y] = !ledOn[x][y];
```

```
}
```

<- Find the column with LED lights on

<- Turn every LED that represents a pipe off. Set alive to false if a the bird happens to collide with the pipe



- The amount of the time the program is stored in the variable called now.
- If the time interval between the last time the pipe was shifted to the left and now is greater than BAR_SHIFT_DELAY, then then the program will shift to the left.
- The third if statement checks to see if the bird has collided with the pipe and if so, the program ends the game

Use of Program Duration

```
x = star_x;

y = i;

//if you are about to turn off a star but then you realize that that is where the bird is increase the amount of stars that the

if (star_x == bird_x and i == 0){

    stars += 1;

    bird_x = star_x;

}else{

    ledOn[x][y] = !ledOn[x][y];

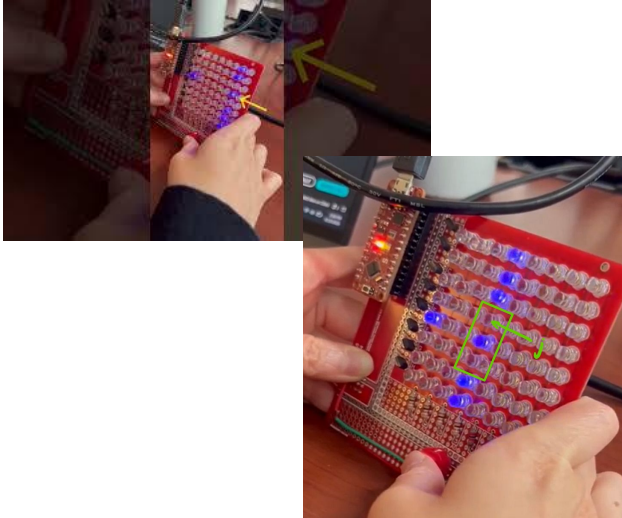
    display(ledOn);

}
```

If the bird happens to be at the same location of the star when the program tries to remove the column instead of removing the star, stars(the number of stars the user obtains through out the game) gets increased by 1

- The program also checks to see if the bird has collided with the star.
- This is so that the program knows to increment stars by one and not turn off the light that represents the bird as well as the star

Use of Randomness



```
//create the first pipe

hole = random(1, 4);

for(byte j = 0; j < hole; j++){

    x = j;

    y = 7;

    ledOn[x][y] = !ledOn[x][y];

}

for(byte j = hole + 3; j < 8; j++){

    x = j;

    y = 7;

    ledOn[x][y] = !ledOn[x][y];

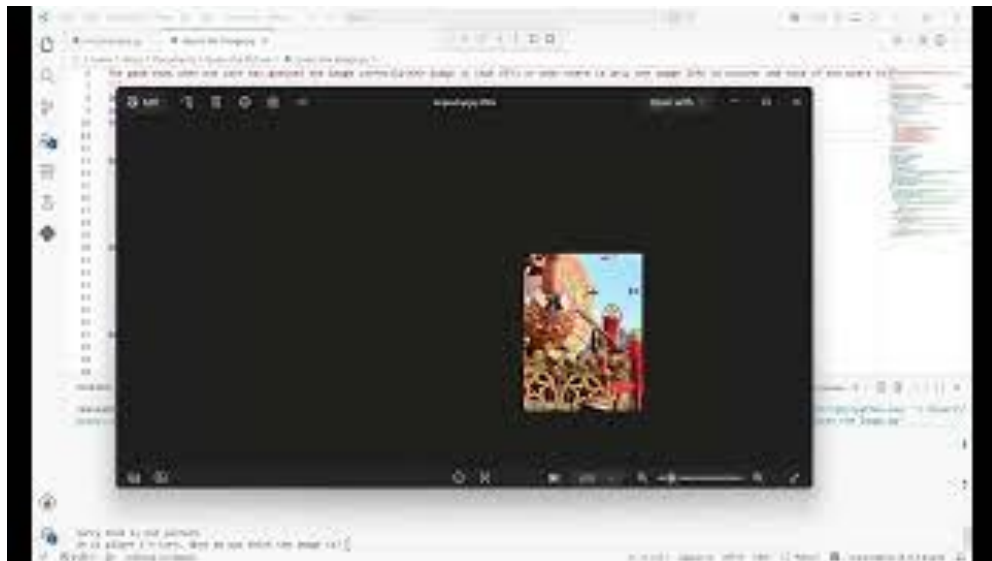
}
```

- Each time a new pipe is generated the pipe hole is in a new random location
- This is done using the random function to generate the row where the hole begins

Guess the Picture with Python

Overview: This code reveals one small section of a picture per round to a user who tries to guess what is depicted in the picture. The code makes an API call to ChatGPT to verify the user's answers. This helps account for the variability of user's responses that may not exactly match the answer given by the coder. The demo on the next page shows a version of the game without the API call because I currently out of tokens which are needed to make the API call.

Game Demo



Using the API Call

```
current_image = Image(pictures[name_image_to_guess])
```

```
def __init__(self, answer):  
    self._correctanswer = answer  
    self._hints = []
```

```
while len(puzzle_pieces_list) != 1 and current_image.get_answer_validation(guess) == False:
```

- Code in Guess the Image.py (main program)
- Code in TypelImage.py

Create an image object by passing in the answer for the current picture in play

Calling the method `get_answer_validation` in `TypelImage.py`

- The class called `TypelImage.py` has a constructor which creates an `Image` object
- The code calls methods on the image object to get responses from ChatGPT

Using the API Call

```
def get_answer_validation(self, guess):
    chat_completion = openai.chat.completions.create(
        messages=[{
            "role": "user",
            "content": "My friends are playing guess the image. The correct answer is " + self._correctanswer + ". Would " + str(guess) + " be
        }],
        model="gpt-4",
    )
    response_str = chat_completion.choices[0].message.content
    if response_str.strip(".").lower() == "yes":
        return True
    return False
```

Return whether or not the user's guess is valid

Correct answer

User guess

Thank You!